Cloud Scalability and Resilience on AWS

What changes when the Product service scales?

Horizontal scaling of the Product-Coupon application

Mark Crowe

MSc in Software Design with Cloud Computing

Container Design and Deployment – CA1

*"~~Peace~~ **Failure** was never an option."* – Magneto, *X-Men: First Class* trailer (2011), adapted

November 2026

Built with Reveal.js, ARJ, Mermaid and ECharts – Source

Download as PDF

© 2026 Mark Crowe

Version 1.8.2

tinyurl.com/3r3c23hm

a00349034.github.io/cloud-scaling-resilience-aws/docs/screencast-deck

About the author



Mark Crowe

Qualifications

- MSc in Software Design with Cloud Computing (2027)
- BSc in Computer Systems

github.com/A00349034

A00349034@outlook.ie

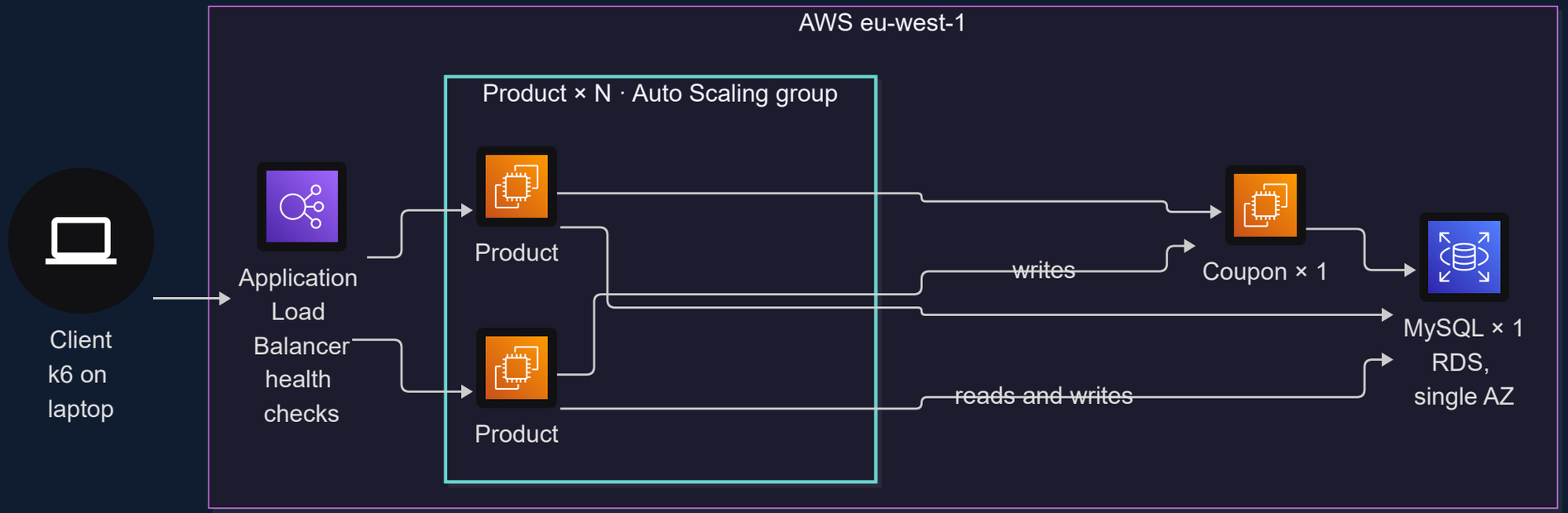
Languages

C# | Java | JavaScript | Python | TypeScript

Frameworks, data and tools

.NET | .NET MAUI | Big Data | Django | Jupyter Notebooks | MySQL | Reveal.js | Spring

One endpoint, shared dependencies



1 – The architecture as tested. Only the Product tier (outlined) changes

- Changed between tests: the number of Product instances (1, 2, or scaled automatically)
- Kept single in every test: Coupon and the database

On the AWS console: one healthy target

1 Total targets	1 Healthy	0 Unhealthy	0 Unused	0 Initial	0 Draining
	0 Anomalous				

▸ **Distribution of targets by Availability Zone (AZ)**
Select values in this table to see corresponding filters applied to the Registered targets table below.

Targets | Monitoring | Health checks | Attributes | Tags

Registered targets (1) Info

Anomaly mitigation: **Not applicable** | Deregister | Register targets

Target groups route requests to individual registered targets using the protocol and port number specified. Health checks are performed on all registered targets according to the target group's health check settings. Anomaly detection is automatically applied to HTTP/HTTPS target groups with at least 3 healthy targets.

Filter targets

1

Instance ID	Name	Port	Zone	Health status	Health status details	Admini...	Overrid...	Launch...	Anomaly detectio...
i-0ca229ce39b54ac77	tus-ca1-product	9090	eu-west-1b (eu...	Healthy	-	No override.	No overrid...	November...	Normal

2 – The load balancer's target group in the elasticity run, before scale-out: one Product instance, healthy. Health checks decide where requests go[1]

A controlled comparison

1 Baseline

One Product instance
Scaling off

2 Compare

Two instances
Same workload and data

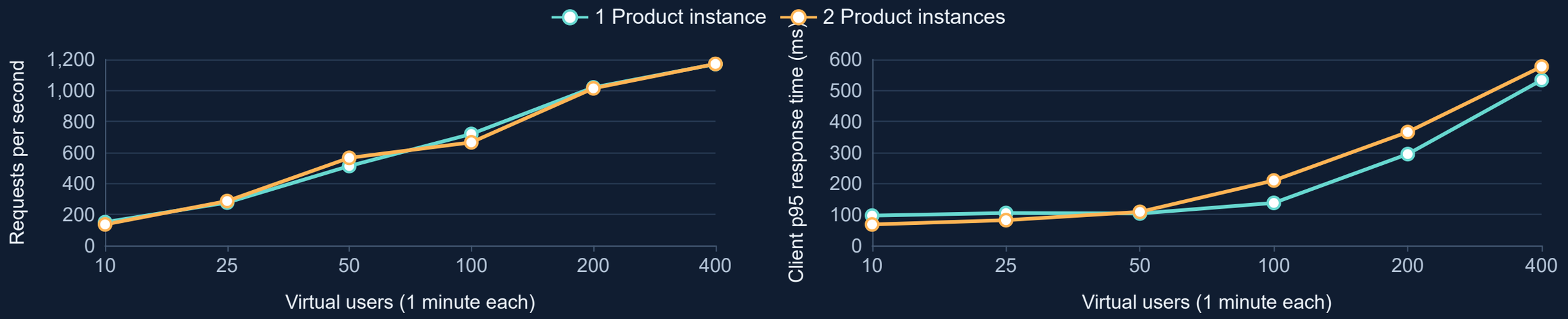
3 Disrupt

Automatic scaling
Terminate an instance

- Measured: requests per second, p95 response time, errors, CPU, event times
- p95, the 95th percentile[9]: 95% of requests finished faster than this time. Client p95 is measured by k6 (a load-testing tool) on the laptop, network included; server p95 by the load balancer, network excluded[6]
- Kept the same: instance type, workload (k6, list products, `GET /productapi/products`), data size (20 products), client location

One Product instance versus two

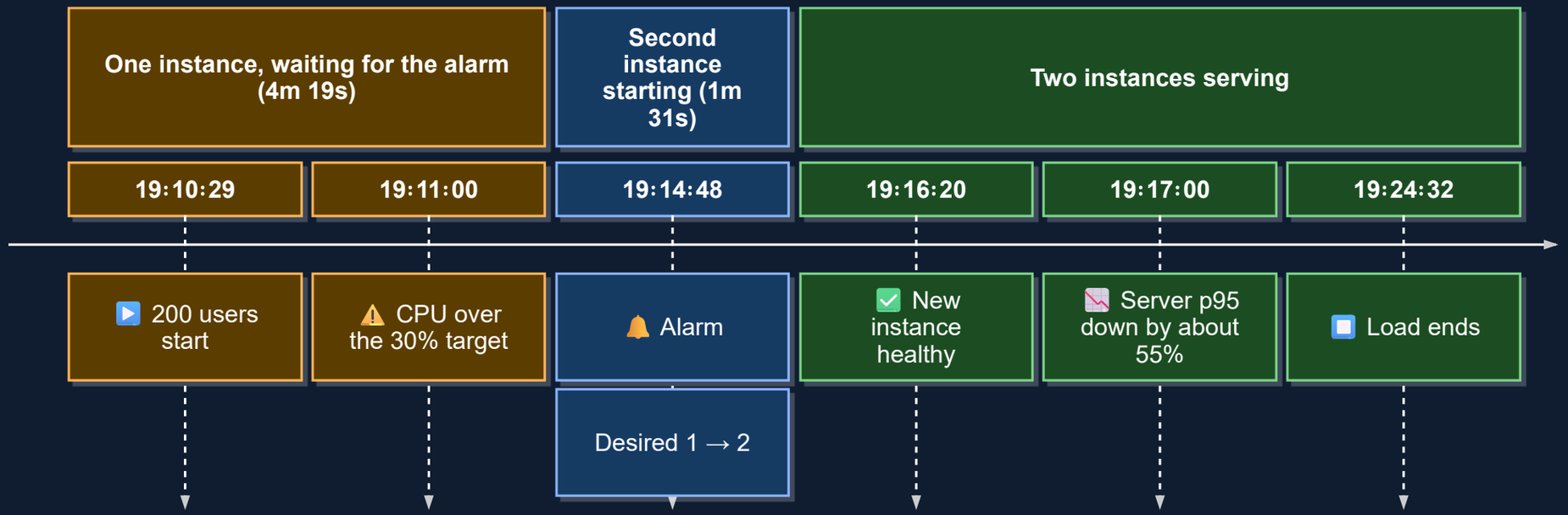
Product instances (200 users)	Requests per second	Client p95	Server p95	Product CPU	Errors
1	1,020	295 ms	45 ms	43%	0.27%
2	1,014	366 ms	29 ms	22% + 20%	0.26%



3 — List products (`GET /productapi/products`) from the laptop, 20 products, 10 to 400 users, 1 minute each

Dataset 1, one-vs-two.csv

New capacity came almost 6 minutes after the load

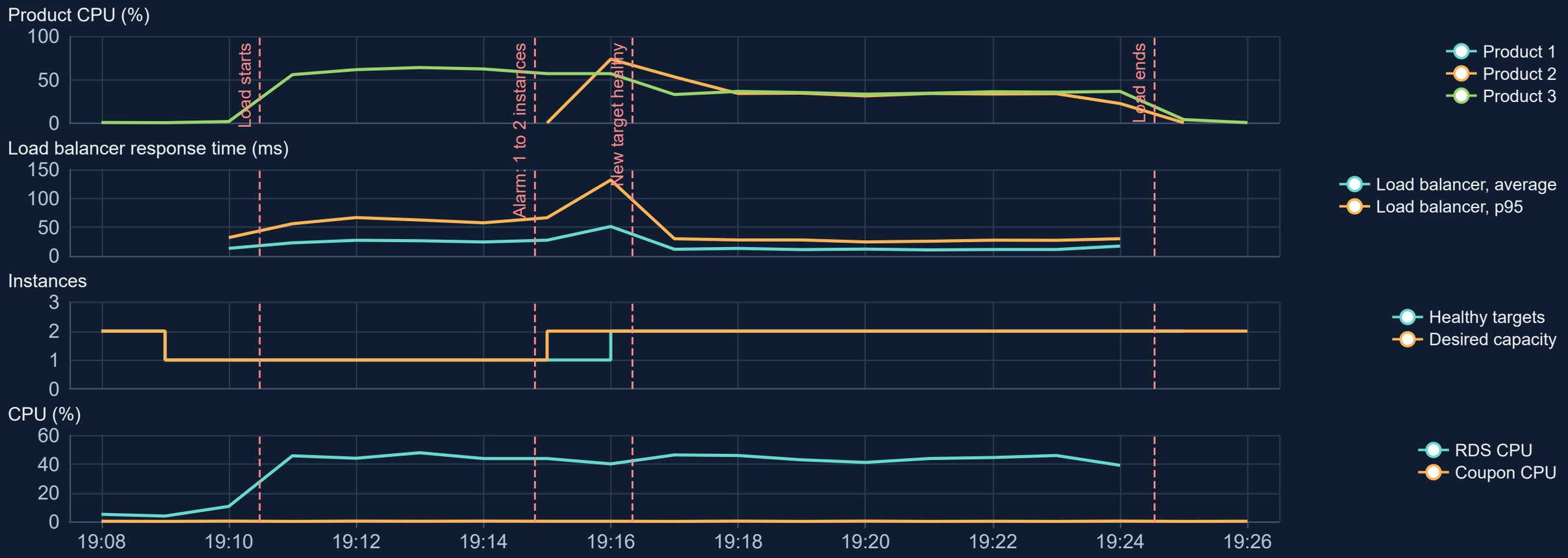


4 – Elasticity events (UTC): 200 users, CPU target 30%[2], min 1, max 2

Dataset 2, elasticity-events.csv

- Almost 6 minutes from the load to useful capacity: 4m 19s waiting for the alarm, 1m 31s for the new instance to start
- The server got faster; the client gained only 7%

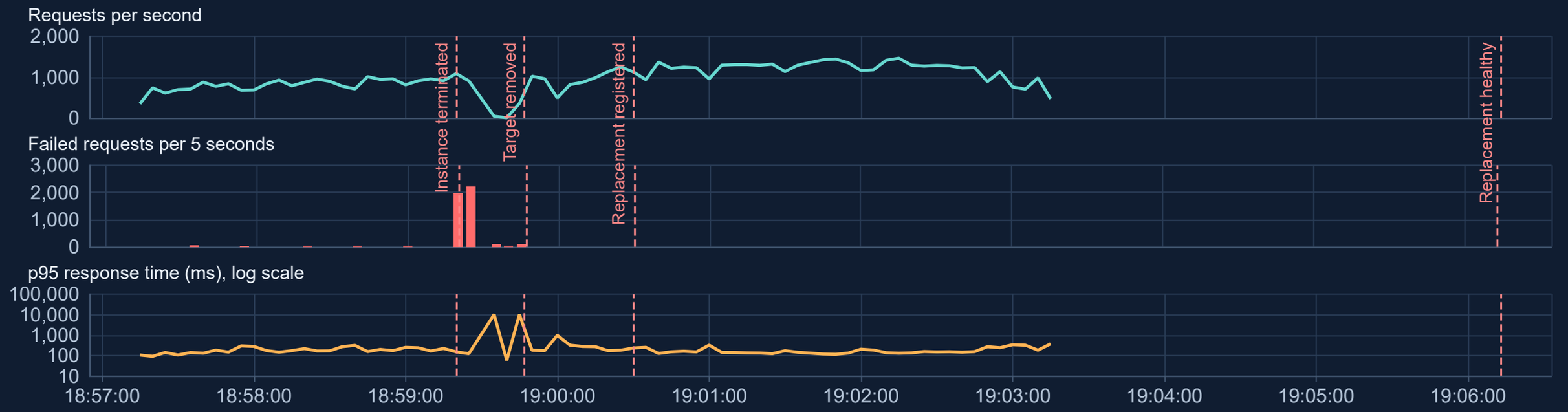
What CloudWatch saw



5 – 1-minute data from Amazon CloudWatch, the AWS monitoring service

Dataset 3, elasticity-cloudwatch.csv

Failure: what did the client see?

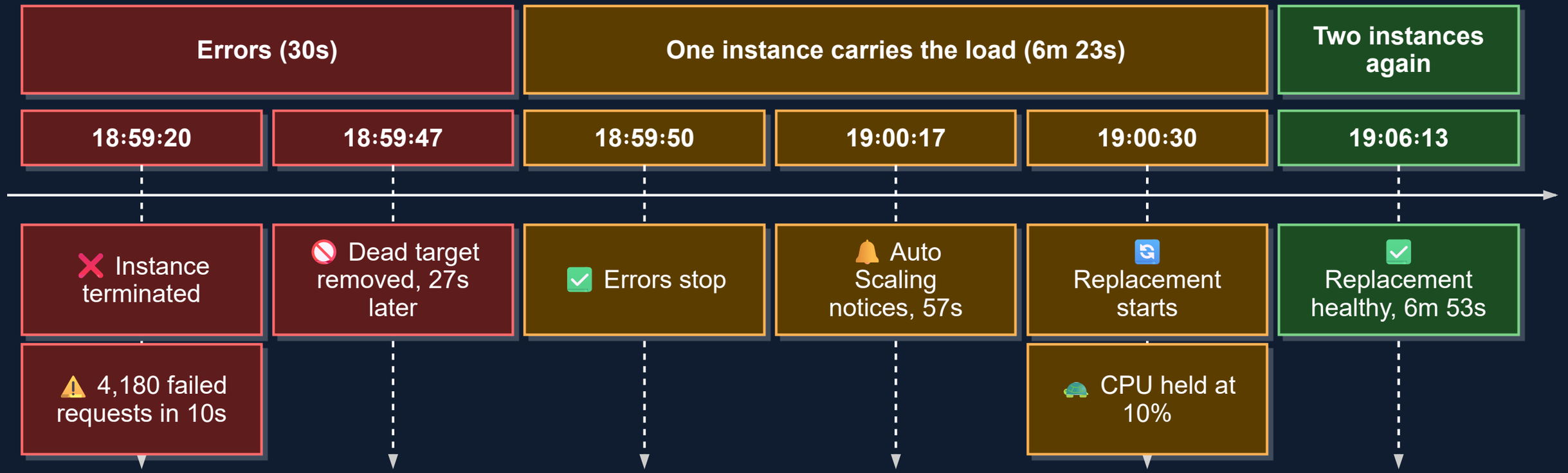


6 – 100 users, 2 instances, one terminated at 18:59:20. k6 requests in 5-second windows

Dataset 4, failure-client-5s.csv

- In the first 10 seconds, there were 4,180 failed requests. Normal on one instance after about 30 seconds 1
- Replacement healthy after 6m 53s: its CPU was held at 10% by standard CPU credits 3

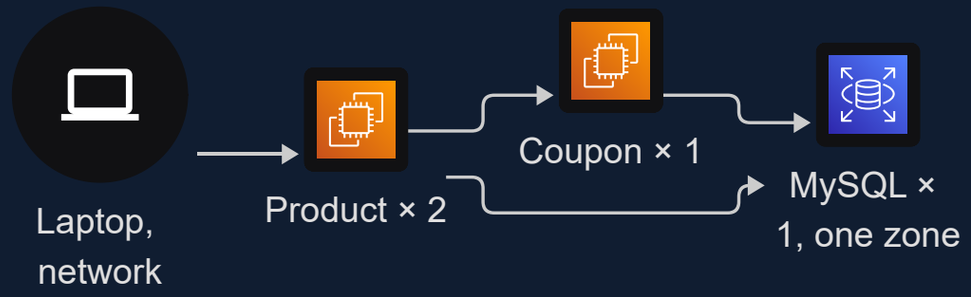
Failure: the events in order



7 – Failure events (UTC): the client recovered in seconds, full capacity took almost 7 minutes

Dataset 5, failure-events.csv

What remains limited?



- My laptop and network: server p95 was only 5-15% of client p95; the mobile network changed my address
- Database: one shared RDS instance in one Availability Zone, CPU up to 48%
- Coupon: one instance on the write path; tested locally, writes fail when it stops
- Start-up: 1.5 minutes with unlimited CPU credits, 6 minutes with standard credits

Above: the parts that stayed single. A single dependency is a risk; it is only a bottleneck where the data shows saturation.

Answer

Scalability

Product CPU halved and server p95 fell 36-55%, but the client saw no gain: my laptop was the limit

Resilience

Survived losing an instance after about 30 s of errors; full capacity back after 7 min

Next improvement

Three tiers across two zones: private subnets, Multi-AZ RDS [4], Coupon in each zone, min 2

Product from a pre-baked image; then run k6 inside AWS

The cost: about USD 32-35 a month for each NAT gateway [5], and more to configure. Details: report, section 5.4

Thanks for coming to my TED Talk

🎤 Questions? ✉ Email me

A00349034@outlook.ie

- The full report: MD report.md · PDF Download the report as a PDF
- References, figures, software and links are below ↓, and → to exit

References

1. Amazon Web Services. Health checks for Application Load Balancer target groups. Elastic Load Balancing User Guide
2. Amazon Web Services. Target tracking scaling policies for Amazon EC2 Auto Scaling. Amazon EC2 Auto Scaling User Guide
3. Amazon Web Services. Unlimited mode for burstable performance instances. Amazon EC2 User Guide
4. Amazon Web Services. Multi-AZ DB instance deployments for Amazon RDS. Amazon RDS User Guide
5. Amazon Web Services. Amazon VPC pricing (NAT gateway)
6. Grafana Labs. k6 documentation: metrics (http_req_duration, p95)
7. Mermaid. Diagrams and charts from text (version 12.3)
8. Apache ECharts. Interactive charts in the browser (version 5.5)
9. Amazon Web Services. CloudWatch statistics definitions: percentiles (p95). Amazon CloudWatch User Guide

Figures

1. The architecture as tested
2. The load balancer's target group on the AWS console
3. One versus two Product instances: requests per second and client p95 against users
4. Elasticity: the events in order
5. Elasticity: Product CPU, response time, capacity and RDS CPU
6. Failure: client requests per second, failed requests and p95
7. Failure: the events in order

The charts are drawn in the browser from ca1-charts.json, made by ca1_chart_data.py from the saved data in test-results. The data behind each figure: Data sets; the notebook make_charts.ipynb shows the numbers and draws the report's charts.

Data sets

Dataset	File and description	Rows	Figure	Open
Dataset 1	one-vs-two.csv One Product instance versus two: requests per second and client p95 by users	12	3	View Download
Dataset 2	elasticity-events.csv Elasticity: the events in order	6	4	View Download
Dataset 3	elasticity-cloudwatch.csv Elasticity: CloudWatch, 1-minute points	134	5	View Download
Dataset 4	failure-client-5s.csv Failure: k6 requests in 5-second windows	72	6	View Download
Dataset 5	failure-events.csv Failure: the events in order	6	7	View Download

View: the data as a table, with what each column means. Download: the CSV file. Made by csv_markdown.py; all data sets: readme.md

Report: report.md · Slides: slides.md · Code: scripts · Evidence: test-results

Software used

Applications

- Docker
- Java 17 (Amazon Corretto)
- Spring Boot 4.1

AWS

- Application Load Balancer
- AWS CLI v2
- CloudWatch
- EC2
- EC2 Auto Scaling
- RDS for MySQL
- Systems Manager

Data and charts

- Jupyter
- matplotlib
- pandas
- Python

Diagrams and slides

- Apache ECharts 5.5
- ARJ 1.8
- Mermaid 12.3
- Reveal.js 5.1

Load testing

- Grafana k6

Scripts

- PowerShell 7

Video

- ffmpeg
- Puppeteer
- Python

Links

- Report: report.md · Download the report as a PDF
- These slides: slides.md, narration speaking-script.md · Download the slides as a PDF
- Code and evidence: github.com/A00349034/cloud-scaling-resilience-aws
- Notes: © 2026 Mark Crowe. AWS service and software icons from Iconify's "logos" pack (CC0) and Devicon (MIT); other icons Material Design Icons (Apache 2.0); site icons belong to their sites; reveal.js © Hakim El Hattab, MIT licence. Generative AI (Claude) helped build the scripts, diagrams and slides; all measurements are from my own AWS deployment

That's all, folks ❗